JAVASCRIPT

SCOPES ET CLOSURES

No notes on this slide.

JAVASCRIPT

Scopes et closures

En programmation les variables ont une *portée* (*scope* en anglais), c'est-à-dire le contexte dans lequel elles existent

```
let condition = true;
if(condition){
  let maValeur = 5;
  // Ici maValeur est accessible
}
console.log(maValeur); // Erreur : maValeur n'existe pas ici
```

JavaScript gère la mémoire automatiquement, les espaces mémoire nécessaires sont automatiquement réservés. Lorsqu'un espace mémoire n'est plus utilisé, JavaScript récupère cet espace.

Récupérer l'espace mémoire non utilisé est une tâche difficile, elle est gérée par une partie dédiée du moteur JavaScript : le *garbage collector*. C'est le cas de la plupart des languages de haut niveau, comme le Python.

Dans les languages dit de "bas niveau", la mémoire est souvent gérée manuellement. C'est le cas par exemple en C/C++. Dans ces cas il n'existe pas de garbage collector, et c'est à la personne qui programme de dire quand un espace mémoire doit être alloué ou désalloué. Cela permet une meilleure précision dans la gestion de la mémoire, mais est souvent la source de beaucoup d'erreurs. Les garbage collectors permettent de ne plus se soucier de cet aspect. En contre-partie, la mémoire n'est pas directement récupérée, mais le garbage collector est exécuté à certains moment pour analyser l'utilisation actuelle de la mémoire et récupérer les zones de mémoire qui ne sont plus utilisées.

JAVASCRIPT

Scopes et closures

Les fonctions JavaScript possèdent également une portée propre

```
function addFive(x){
  const five = 5;
  return x + five;
}

const result = addFive(4); // Ligne valide
const result2 = 4 + five;  // Erreur : five n'existe que dans la fonction
```

Toute déclaration dans une fonction n'existe que dans cette fonction

No notes on this slide.

JAVASCRIPT

Scopes et closures

Les variables déclarées en dehors de la fonction peuvent être utilisées à l'intérieur

```
let outsideVariable = 5;

function add(x){
  return x + outsideVariable;
}

console.log(add(4)); // 9
outsideVariable = 10;
console.log(add(4)); // 14
```

La fonction garde *une référence* vers son contexte. C'est-à-dire que changer une valeur utilisée dans la fonction aura un impacte.

No notes on this slide.

JAVASCRIPT

Scopes et closures

Que se passe-il dans ce cas ? (Exemple tiré de developer.mozilla.org)

```
function makeFunc() {
  const name = "Mozilla";
  function displayName() {
    console.log(name);
  }
  return displayName;
}
const myFunc = makeFunc();
myFunc(); // Que fait cette ligne ?
```

1. Affiche "Mozilla"
2. Retour une erreur, "name" n'est pas définit
3. Affiche *undefined*

No notes on this slide.

JAVASCRIPT

Scopes et closures

Que se passe-il dans ce cas ? (Exemple tiré de developer.mozilla.org)

```
function makeFunc() {
  const name = "Mozilla";
  function displayName() {
    console.log(name);
  }
  return displayName;
}
const myFunc = makeFunc();
myFunc(); // Que fait cette ligne ?
```

Ce script affichera "Mozilla". En JavaScript, les fonctions retiennent tout leur contexte, même si ce contexte devient hors de portée. Ici, bien que `name` soit déclaré dans la fonction, et que `name` soit hors de portée lors de l'appel à `myFunc()`, la fonction a tout de même conservé son contexte, et donc la valeur de `name`

Les fonctions en JavaScript sont donc la combinaison de leurs instructions et de leur contextes. On dit alors que les fonctions forment des *fermetures* (*closures* en anglais).

No notes on this slide.

JAVASCRIPT

Scopes

Site officiel :

https://developer.mozilla.org/en-US/docs/Glossary/Scope

https://developer.mozilla.org/fr/docs/Web/JavaScript/Guide/Closures

No notes on this slide.